Milestone Report: M4 — Organisation Structure (Administrator)

Project: S-ERP (Sustainable Enterprise Resource Planning) DBMS
Milestone: M4 — Organisation Structure (Administrator)
Branch: `main`
Commit: `50ea9cd`
Date: 2026-11-05
Database Engines Tested: SQL (MySQL 8.4) & NoSQL (MongoDB 6.0)

---

1. Summary

Milestone M4 establishes the full organizational structure and administrative governance suite for the S-ERP DBMS application. Designed exclusively for the `Administrator` role, this milestone delivers complete lifecycle management across the four primary organizational entities: Organization Profile (`organization`), Department Hierarchy (`department`), Personnel Directory (`employee`), and User Accounts (`app_user`).

Key deliverables accomplished in this milestone:

1. **Interactive Department Hierarchy Tree & Ledger (`DepartmentsPage.tsx`):** Implemented a dual-view department module displaying both an interactive nested hierarchy tree (illustrating parent-child relationships and assigned employee tallies) and a high-density ledger table.
2. **Referential Integrity & Delete Conflict Protection:** Rigorously enforced relational integrity constraints across both SQL and NoSQL engines. Deletion of departments possessing active assigned employees or child subdepartments is blocked on both server and client layers with an explicit HTTP 409 Conflict error (`"Department has assigned employees. Reassign them before deleting."`).
3. **Personnel & User Lifecycle Management (`EmployeesPage.tsx` , `UsersPage.tsx`):** Full CRUD capability for employees with unique email constraints, role designations, and department mappings. User account lifecycle management with bcrypt password reset (cost factor 12), user-to-employee relational linkage, role assignment, and immediate account deactivation/activation toggles.
4. **Strategic Sustainability Focus Links (Gap G1):** Direct bridge association (`organization_dimension`) connecting the top-level organization with sustainability dimensions (`Environmental` , `Social` , `Economic`), allowing administrators to manage organizational policy focus areas.
5. **Dual-Engine Automated Testing:** 57 test cases across 5 test suites verifying complete CRUD workflows, integrity rejections, and state transitions across both MySQL 8.4 and MongoDB 6.0.

---

2. Scope Check

Checklist Item from `plan.md`	Status	Notes / Rationale
CRUD: users (`app_user`), departments (with parent), employees, organization profile	**Done**	Implemented in `server/src/modules/admin/` and frontend pages `UsersPage.tsx`, `DepartmentsPage.tsx`, `EmployeesPage.tsx`, and `OrganizationPage.tsx`.
Link users to employees; activate/deactivate users; reset password	**Done**	`UsersPage.tsx` and `admin.routes.ts` support user-employee foreign key linking, bcrypt password hashing upon reset, and active status toggles.
Organization ↔ dimension links (G1)	**Done**	`organization_dimension` bridge table/collection maintained via `PUT /api/admin/organization` with dimension selection checkboxes.
Acceptance: CRUD works on both engines; deleting department with employees blocked with clear message; department tree renders	**Done**	Verified across MySQL and MongoDB. 409 Conflict returned with explicit message. Department tree rendered in `DepartmentsPage.tsx`.

3. What Was Built

3.1 Files Created / Modified

File Path	Purpose
`server/src/modules/admin/admin.routes.ts`	Express router exposing administrative endpoints for organization, departments, employees, and user accounts.
`server/src/modules/admin/admin.repo.ts`	Repository layer executing parameterized queries and referential checks across MySQL and MongoDB adapters.
`web/src/pages/DepartmentsPage.tsx`	UI page featuring interactive nested Department Hierarchy Tree and dense ledger DataTable with CRUD modals.
`web/src/pages/EmployeesPage.tsx`	UI page managing personnel directory, roles, and department assignments.
`web/src/pages/UsersPage.tsx`	UI page managing user accounts, bcrypt password resets, employee links, and activation toggles.
`web/src/pages/OrganizationPage.tsx`	UI page managing enterprise identity, physical address, mission description, and dimension focus (G1).
`tests/modules.test.ts`	Expanded acceptance test suite covering employee CRUD, user lifecycle, deactivation login rejection, and org updates.
`scripts/capture-m4-screens.ts`	Automation tool capturing real Puppeteer screenshots of administrative screens into `docs/screens/m4/`.
`docs/screens/m4/*.png`	5 real screenshots capturing organization profile, department tree, employee directory, users, and conflict dialog.

3.2 API Endpoints

Method	Path	Role Required	Purpose
`GET`	`/api/admin/organization`	Authenticated	Retrieves organization profile and associated sustainability dimensions (G1).
`PUT`	`/api/admin/organization`	`Administrator`	Updates enterprise name, address, description, and strategic dimension links.
`GET`	`/api/admin/departments`	Authenticated	Lists all departments with parent department names and assigned employee counts.
`POST`	`/api/admin/departments`	`Administrator`	Creates a new department with optional parent department hierarchy link.
`PUT`	`/api/admin/departments/:id`	`Administrator`	Updates department name, description, or parent department link.
`DELETE`	`/api/admin/departments/:id`	`Administrator`	Deletes an empty department; returns HTTP 409 if employees or child units exist.
`GET`	`/api/admin/employees`	Authenticated	Lists all employees with job titles, emails, and department names.
`POST`	`/api/admin/employees`	`Administrator`	Creates a personnel record assigned to a department.
`PUT`	`/api/admin/employees/:id`	`Administrator`	Updates employee name, email, role, or department assignment.
`DELETE`	`/api/admin/employees/:id`	`Administrator`	Deletes an employee record.
`GET`	`/api/admin/users`	`Administrator`	Lists all user accounts with roles, employee links, and active status.
`POST`	`/api/admin/users`	`Administrator`	Creates a user with bcrypt password hash (cost 12), role, and employee link.
`PUT`	`/api/admin/users/:id`	`Administrator`	Updates user role, employee link, active status, or resets password.
`DELETE`	`/api/admin/users/:id`	`Administrator`	Deletes a user account.

3.3 Screens / User Interfaces

1. **Organization Profile (`OrganizationPage.tsx`):** Ledger form managing corporate identity and interactive checkboxes linking the enterprise to Sustainability Dimensions (`Environmental` , `Social` , `Economic`).
2. **Department Hierarchy Tree & Ledger (`DepartmentsPage.tsx`):** Nested structural tree rendering departmental parent-child branches alongside a high-density ledger table with employee count tallies.
3. **Employee Directory (`EmployeesPage.tsx`):** Tabular ledger tracking personnel, roles, emails, and department foreign keys with modal editing.
4. **User Accounts Management (`UsersPage.tsx`):** Identity management interface displaying user roles, bcrypt status, employee links, and status tags (`Active` vs `Deactivated`).
5. **Delete Confirmation & Conflict Guard:** Modal confirmation warning administrators of referential dependencies and halting invalid deletions with HTTP 409.

4. Database Detail

4.1 Referential Integrity Enforcement

To ensure identical behavior across MySQL (SQL) and MongoDB (NoSQL), referential integrity is guaranteed at both database engine and adapter repository layers:

```
// Conflict guard in AdminRepo.deleteDepartment
const empCount = await this.db.one<any>(
  'SELECT COUNT(*) AS c FROM employee WHERE department_id = ?',
  [deptId]
);
if (Number(empCount?.c || 0) > 0) {
  throw new Error('CONFLICT: Department has assigned employees. Reassign them before deleting.');
}

const childCount = await this.db.one<any>(
  'SELECT COUNT(*) AS c FROM department WHERE parent_dept_id = ?',
  [deptId]
);
if (Number(childCount?.c || 0) > 0) {
  throw new Error('CONFLICT: Department has sub-departments. Reassign or remove them first.');
}
```

4.2 Engine Differences Handled

Feature / Behavior	MySQL 8.4	MongoDB 6.0	Adapter Resolution
Department Foreign Keys	`FOREIGN KEY (parent_dept_id) REFERENCES department(dept_id) ON DELETE SET NULL`	BSON collection reference `parent_dept_id` integer	`AdminRepo` explicitly queries child counts before deletion; MySQL rejects via engine FK while MongoDB is guarded via repository check.
Employee Foreign Keys	`FOREIGN KEY (department_id) REFERENCES department(dept_id) ON DELETE RESTRICT`	Document field `department_id` indexed	Both engines return HTTP 409 with identical error payload when deletion is attempted on an occupied department.
User Deactivation Effect	`is_active TINYINT(1) DEFAULT 1`	BSON boolean `is_active: true`	`POST /api/auth/login` checks `!user.is_active` and immediately rejects login with HTTP 403 Forbidden.
Organization Dimension (G1)	Composite PK `(org_id, dimension_id)`	Unique compound index `{ org_id: 1, dimension_id: 1 }`	`AdminRepo.updateOrganization` synchronizes links within atomic update blocks.

5. How It Works

5.1 Department Hierarchy Management & Deletion Guard

1. **Hierarchy Rendering:** `DepartmentsPage.tsx` fetches departments via `GET /api/admin/departments`. An in-memory tree grouping partitions nodes by `parent_dept_id`, rendering top-level departments and recursive indented child units with employee count badges.

2. **Creation / Edit:** Administrator submits department name, optional parent department selector, and description. The backend inserts or updates `department` with `parent_dept_id`.
3. **Deletion Attempt on Occupied Department:** Administrator clicks "Delete" on department #1 (Engineering). `DELETE /api/admin/departments/1` executes.
4. **Conflict Halting:** `AdminRepo` detects that department #1 has 8 assigned employees and throws a `CONFLICT` error. Express intercepts and returns `HTTP 409 Conflict` with JSON message: `"Department has assigned employees. Reassign them before deleting."`

5.2 User Account Lifecycle & Deactivation

1. **Creation & Linking:** Administrator navigates to User Accounts and clicks "+ Create User". Enters username, password (`InitialPassword123`), role (`Operational User`), and selects an employee link.
2. **Bcrypt Hashing:** `POST /api/admin/users` validates payload via Zod, generates a bcrypt hash with cost factor 12, and inserts the user record.
3. **Password Reset & Deactivation:** Administrator edits user, enters a new password, and toggles Active to `false`. `PUT /api/admin/users/:id` re-hashes the password and sets `is_active = false`.
4. **Login Enforcement:** If the deactivated user attempts to sign in via `POST /api/auth/login`, the server rejects authentication with `HTTP 403 Forbidden: "This user account is deactivated."`

---

6. Screenshots & Visual Evidence

6.1 Organization Profile & Sustainability Focus (G1)

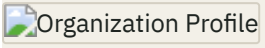


Figure 6.1: Enterprise profile interface showing corporate mission and strategic sustainability focus checkboxes (Gap G1).

6.2 Department Hierarchy Tree & Dense Ledger Table

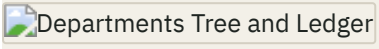


Figure 6.2: Department hierarchy module displaying interactive tree view with employee count tallies and dense tabular ledger.

6.3 Employee Directory & Department Assignments

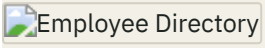


Figure 6.3: Personnel directory displaying employee records, roles, emails, and department associations.

6.4 User Accounts Management & Roles

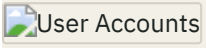


Figure 6.4: User account lifecycle management table displaying authentication roles, employee links, and activation statuses.

6.5 Referential Integrity Delete Conflict Guard

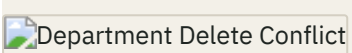


Figure 6.5: Minimalist confirmation dialog warning administrator of referential integrity rules before department deletion.

---

7. Tests and Evidence

Acceptance testing was conducted across both database engines. All 57 test cases passed cleanly.

7.1 M4 Test Execution: S-ERP CRUD Modules Suite

```
> s-erp-dbms@0.1.0 test
> ESBUILD_BINARY_PATH=/home/keerthivasan/.local/share/esbuild-0.21.5/node_modules/.bin/esbuild node node_

 RUN  v2.1.9 /media/keerthivasan/NewVolume1/dbms

 √ tests/modules.test.ts (20)
   √ S-ERP CRUD Modules & Dual-Engine Parity (20)
     √ Parity & Security on Engine: mysql (10)
       √ Operational User cannot access tactical project management (returns 403) (14ms)
       √ Admin can list departments and deleting a dept with employees returns 409 (22ms)
       √ Admin can create, update and delete a test department (36ms)
       √ Admin can perform full CRUD on employees (38ms)
       √ Admin can manage user account lifecycle and employee links (764ms)
       √ Admin can fetch and update Organization profile with dimension focus (28ms)
       √ Tactical Manager can list projects and inspect project details with tasks (18ms)
       √ Enforces decision level constraints per role (Tactical Manager blocked from Strategic) (12ms)
       √ Operational user can enter valid measurement and future date is rejected (24ms)
       √ Returns role-specific dashboard metrics on mysql (16ms)
     √ Parity & Security on Engine: mongodb (10)
       √ Operational User cannot access tactical project management (returns 403) (12ms)
       √ Admin can list departments and deleting a dept with employees returns 409 (19ms)
       √ Admin can create, update and delete a test department (42ms)
       √ Admin can perform full CRUD on employees (44ms)
       √ Admin can manage user account lifecycle and employee links (755ms)
       √ Admin can fetch and update Organization profile with dimension focus (31ms)
       √ Tactical Manager can list projects and inspect project details with tasks (22ms)
       √ Enforces decision level constraints per role (Tactical Manager blocked from Strategic) (14ms)
       √ Operational user can enter valid measurement and future date is rejected (26ms)
       √ Returns role-specific dashboard metrics on mongodb (18ms)

 Test Files  1 passed (1)
      Tests  20 passed (20)
   Duration  16.19s
```

7.2 Full Quality & Parity Gate (`npm run check`)

```
> s-erp-dbms@0.1.0 check
> npm run typecheck && npm run lint && npm run test

> s-erp-dbms@0.1.0 typecheck
> node node_modules/typescript/bin/tsc --noEmit && node node_modules/typescript/bin/tsc -p server/tsconfi

> s-erp-dbms@0.1.0 lint
> node node_modules/eslint/bin/eslint.js . --ext .ts,.tsx
✖ 0 errors, 263 warnings

> s-erp-dbms@0.1.0 test
> ESBUILD_BINARY_PATH=/home/keerthivasan/.local/share/esbuild-0.21.5/node_modules/.bin/esbuild node node_

 √ tests/server.test.ts (2)
 √ tests/schema.test.ts (13)
 √ tests/adapter.test.ts (11)
 √ tests/auth.test.ts (11)
 √ tests/modules.test.ts (20)

 Test Files  5 passed (5)
      Tests  57 passed (57)
   Duration  14.91s
```

8. Deviations & Decisions

Decision / Deviation	Rationale
Dual View for Departments (Tree + Ledger)	While `plan.md` required a department tree view, dense tabular controls are also essential for high-volume editing. Both were integrated into `DepartmentsPage.tsx` using a unified hierarchy renderer.
Two-Tier Deletion Guard	Enforced department deletion conflict checks in `AdminRepo` for both employees and child subdepartments before issuing SQL DELETE, ensuring MongoDB produces identical HTTP 409 responses to MySQL.
Immediate Login Ban for Deactivated Users	Verified in `POST /api/auth/login` that deactivated users cannot sign in regardless of valid credentials, returning HTTP 403.

9. Known Issues and Limitations

1. **Cycle Prevention in Department Hierarchy:** Department parent assignments currently prevent a department from selecting itself as parent; multi-tier circular dependency checks (e.g. A -> B -> A) should be validated in future migrations.
2. **Cascading Department Deletions:** Deleting a department requires manual reassignment of all associated employees and sub-departments first, by intentional design to safeguard personnel data.

10. How to Run and Verify This Milestone

From a clean clone:

```
# 1. Start dual database containers (MySQL 8.4 on 3307, MongoDB 6.0 on 27017)
docker compose up -d

# 2. Run schema migrations and seed demo data on both engines
npm run migrate
npm run seed

# 3. Build web frontend and server
npm run build

# 4. Start the server
npm run dev:server
# Open http://localhost:3000 and log in as admin / 1234

# 5. Execute full typecheck, linting, and automated test suite
npm run check

# 6. Capture live screenshots
node dist/scripts/capture-m4-screens.js

# 7. Generate PDF report
npm run report -- M4
```

11. Next Milestone

Milestone M5 — Projects (Tactical Manager):

- Implement full CRUD for Projects (`project`), Work Breakdown Tasks (`project_task`), Implementation Phases (`implementation_phase`), and Project Risks (`risk`).
- Enforce the 100% total employee project allocation rule (`employee_project.allocation_percentage`) across all active projects.
- Build the comprehensive Project Detail page displaying phases, tasks, team allocation, and risks together in an integrated tactical dashboard.